

EXPERIMENT 1:

Aim

To write a Python program that solves the 8-puzzle problem using the A* search algorithm.

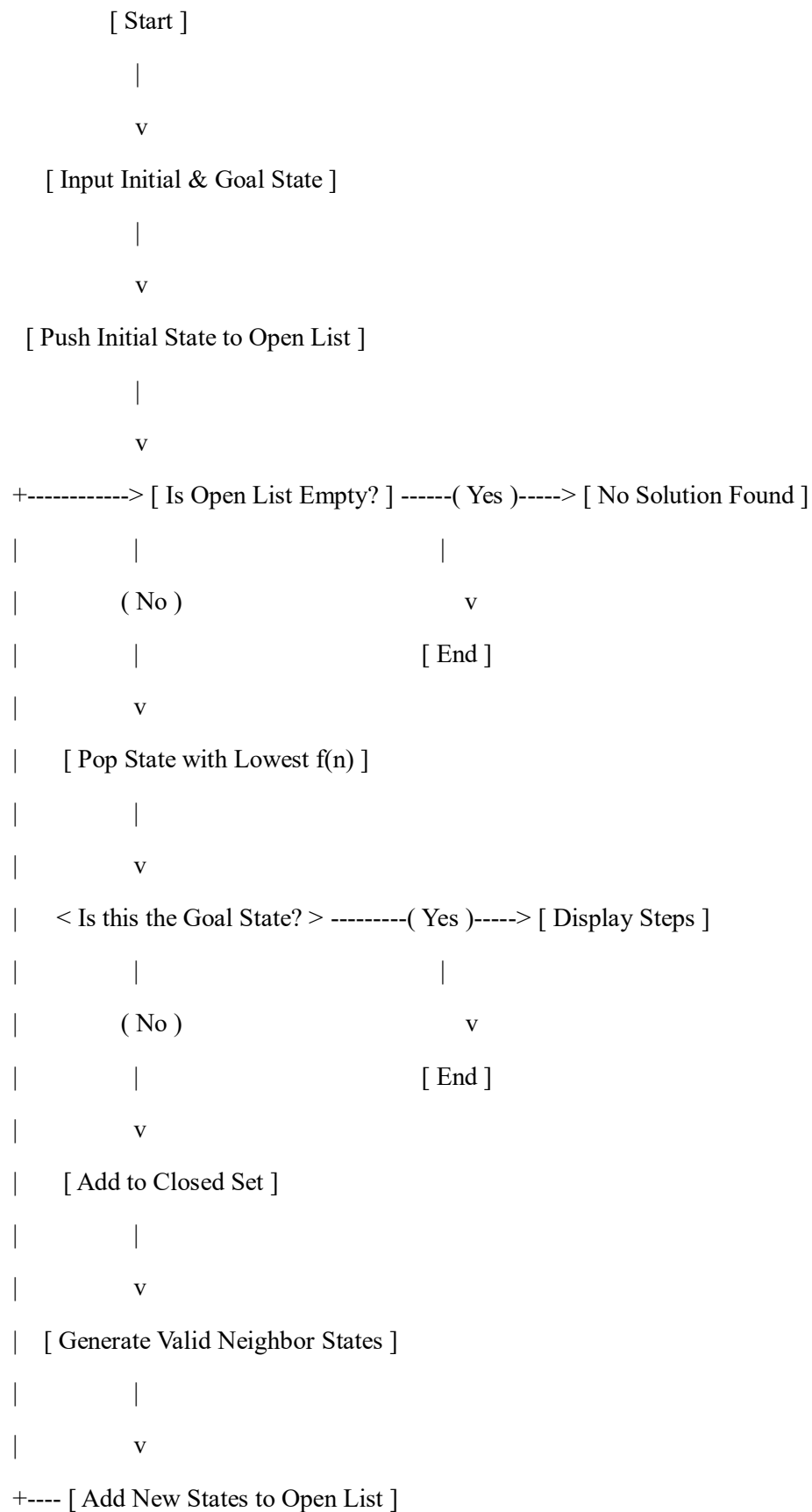
Objective

- To represent the 8-puzzle board and simulate moving the blank tile (UP, DOWN, LEFT, RIGHT).
- To implement the A* algorithm using Manhattan distance to find the shortest path to the goal state.
- To print each step showing how the puzzle is solved.

Algorithm

1. Set up the target goal state (1 to 8, with 0 as the blank space).
2. Create an open_list (priority queue) for states to explore and a closed_set to keep track of visited states.
3. For the current board, calculate the **Manhattan Distance** (how far each tile is from its target spot).
4. Calculate total cost: $f(n) = g(n) + h(n)$, where:
 - $g(n)$ = moves made so far.
 - $h(n)$ = Manhattan distance heuristic.
5. Push the starting board into open_list.
6. Loop until open_list is empty:
 - Pick the state with the lowest $f(n)$ value.
 - If it matches the goal state, trace back the steps and stop.
 - Otherwise, mark it as visited.
 - Generate all possible moves by sliding the blank tile (0) up, down, left, or right.
 - Add new, unvisited states to open_list.

Flowchart



Code

```
import heapq
```

```
GOAL_STATE = (  
    (1, 2, 3),  
    (4, 5, 6),  
    (7, 8, 0)  
)
```

```
class PuzzleNode:
```

```
    def __init__(self, board, parent=None, move="", g=0):  
        self.board = board  
        self.parent = parent  
        self.move = move  
        self.g = g # Path cost  
        self.h = self.get_manhattan_distance()  
        self.f = self.g + self.h # Total estimated cost
```

```
    def get_manhattan_distance(self):
```

```
        distance = 0
```

```
        targets = {
```

```
            1: (0, 0), 2: (0, 1), 3: (0, 2),
```

```
            4: (1, 0), 5: (1, 1), 6: (1, 2),
```

```
            7: (2, 0), 8: (2, 1), 0: (2, 2)
```

```
        }
```

```
        for r in range(3):
```

```
            for c in range(3):
```

```
                val = self.board[r][c]
```

```
                if val != 0:
```

```
                    tr, tc = targets[val]
```

```

        distance += abs(r - tr) + abs(c - tc)

    return distance

def __lt__(self, other):
    return self.f < other.f

def find_blank(self):
    for r in range(3):
        for c in range(3):
            if self.board[r][c] == 0:
                return r, c

def get_neighbors(self):
    neighbors = []
    r, c = self.find_blank()

    moves = {
        "UP": (r - 1, c),
        "DOWN": (r + 1, c),
        "LEFT": (r, c - 1),
        "RIGHT": (r, c + 1)
    }

    for move_name, (nr, nc) in moves.items():
        if 0 <= nr < 3 and 0 <= nc < 3:
            new_board = [list(row) for row in self.board]
            new_board[r][c], new_board[nr][nc] = new_board[nr][nc], new_board[r][c]
            board_tuple = tuple(tuple(row) for row in new_board)
            neighbors.append(PuzzleNode(board_tuple, self, move_name, self.g + 1))

    return neighbors

```

```

def solve(initial_board):
    start_node = PuzzleNode(initial_board)
    open_list = []
    heapq.heappush(open_list, start_node)
    closed_set = set()

    while open_list:
        current = heapq.heappop(open_list)

        if current.board == GOAL_STATE:
            path = []
            while current:
                path.append(current)
                current = current.parent
            return path[::-1]

        closed_set.add(current.board)

        for neighbor in current.get_neighbors():
            if neighbor.board in closed_set:
                continue
            heapq.heappush(open_list, neighbor)

    return None

def print_board(board):
    for row in board:
        print(" ".join(str(tile) if tile != 0 else "_" for tile in row))

```

```
print()

if __name__ == "__main__":
    # 0 represents the blank space
    start_board = (
        (1, 2, 3),
        (0, 4, 6),
        (7, 5, 8)
    )
    print("Initial State:")
    print_board(start_board)

    solution = solve(start_board)
    if solution:
        print(f"Solved in {len(solution) - 1} moves!\n")
        for step, node in enumerate(solution):
            if step > 0:
                print(f"Step {step}: Move Blank {node.move}")
                print_board(node.board)
    else:
        print("No solution exists for this configuration.")
```

Output

Initial State:

1 2 3

_ 4 6

7 5 8

Solved in 3 moves!

Step 1: Move Blank RIGHT

1 2 3

4 _ 6

7 5 8

Step 2: Move Blank DOWN

1 2 3

4 5 6

7 _ 8

Step 3: Move Blank RIGHT

1 2 3

4 5 6

7 8 _

Result

The 8-puzzle program was executed successfully. Using the A* algorithm, it found the shortest sequence of moves to reach the goal state from the given start state.

Conclusion

The A* search algorithm effectively solves the 8-puzzle problem. By combining actual path cost with a heuristic estimation (Manhattan Distance), it explores fewer unnecessary paths than standard algorithms like BFS or DFS, making it both fast and reliable.